Spring Framework

Spring Web (REST) – Autres outils

Rappel des objectifs du cours

Montrer quelques outils supplémentaires

Présenter d’autres concepts gravitant autour de REST

Sommaire

- **Partie 1** – @RequestParam
- **Partie 2** – Listes Paginées
- **Partie 3** – @ResponseStatus
- **Partie 4** – ResponseEntity
- **Partie 5** – Les DTOs
- **Partie 6** – Les tâches cron
- **Partie 7** – Fichier application.properties

Spring Framework – REST Partie Bonus

Partie 1 – Passer des paramètres dans les requêtes HTTP (@RequestParam)

@RequestParam

On peut parfois trouver des paramètres sur une URL :

HTTP GET

```
http://localhost:8080/api/person?pageSize=25&pageNumber=10
```

@RequestParam

En java, pour extraire ces paramètres :

- **@RequestParam** sur les arguments

HTTP GET

```
http://localhost:8080/api/toto?test=Jean
```

```
@GetMapping("/api/toto")
public String getToto(@RequestParam("test") String blabla) {
    return "Le paramètre est " + blabla;
}
```

On demande un paramètre « test » dans l'URL.

@RequestParam

- **@RequestParam** : rendre optionnel

```
@GetMapping("/api/toto")
public String getToto(
        @RequestParam(value = "test", required = false) String blabla) {
    return "Le paramètre est " + blabla;
}
```

- **@RequestParam** : valeur par défaut

```
@GetMapping("/api/toto")
public String getToto(
        @RequestParam(value = "test", defaultValue = "Jean") String blabla) {
    return "Le paramètre est " + blabla;
}
```

@RequestParam

Différences avec @PathVariable ?

Au niveau du « sens » :

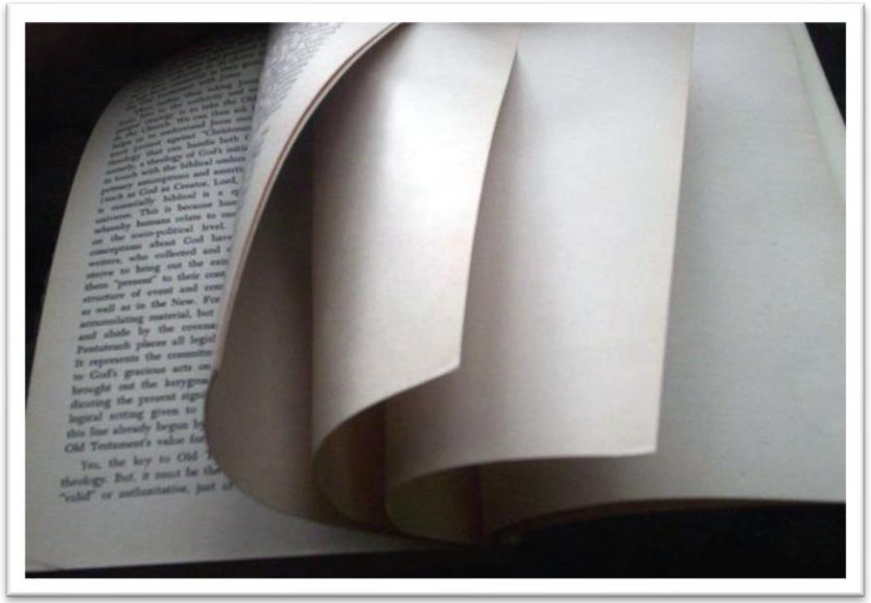
- ***@RequestParam*** est comme un « extra » sur l'URL.
- ***@PathVariable*** fait vraiment partie de l'URL

- **Pour REST**, un lien vers une ressource a une URL unique (comme un chemin vers un fichier) : @PathVariable
- Lequel utiliser ? Dépend du design de notre application

Spring Framework – REST Partie Bonus

Partie 2 – Les listes paginées

Listes Paginées



- Une liste découpée en “pages”
- Chaque page contient **un nombre fixe d’éléments**
- Cela permet de **récupérer des tronçons** plutôt que tous les éléments d’un coup.

Liste Paginées

Dans Spring :

- Objets issus de **`org.springframework.data.domain.*`**

- Page<T>
- Pageable
- PageRequest

Liste Paginées

Page<T>

- Représente notre collection de données
- C'est le "tronçon" de liste
- Générique (comme List<T>), il faut la typer

On l'utilise comme type de retour (ici dans le Service)

Service

```
public Page<Person> findAll(Pageable pageable) {
    return this.personRepository.findAll(pageable);
}
```

Liste Paginées

Pageable

- L'information sur la pagination
- On s'en sert pour **désigner un "format" de page**
- Contient : Numéro de page, taille de page et tri (optionnel)

On l'utilise comme argument pour notre repository

Service

```
public Page<Person> findAll(Pageable pageable) {
    return this.personRepository.findAll(pageable);
}
```

Liste Paginées

PageRequest.of(..)

- Méthode **of** pour construire un *Pageable*
- Permet aussi de mettre un tri

Exemple d'une méthode avec des valeurs « en dur » :

Service

```
public Page<Person> findFirstPageSize25() {
    Pageable pageable = PageRequest.of(0, 25);
    return this.personRepository.findAll(pageable);
}
```

Différentes signatures de « of » :

```
of(int page, int size)                                          PageRequest
of(int page, int size, Sort sort)                               PageRequest
of(int page, int size, Direction direction, String... properties)  PageRequest
ofSize(int pageSize)                                            PageRequest
```

Liste Paginées

Utiliser des listes paginées :

- Méthode **findAll** d'un repository Spring (JpaRepository ou PagingAndSortingRepository)

Extrait de l'interface PagingAndSortingRepository de Spring

Repository

```
/**
 * Returns a {@link Page} of entities meeting the paging restriction provided in the {@link Pageable} object.
 *
 * @param pageable the pageable to request a paged result, can be {@link Pageable#unpaged()}, must not be
 *          {@literal null}.
 * @return a page of entities
 */
Page<T> findAll(Pageable pageable);
```

Liste Paginées

Utiliser des listes paginées :

- Type de retour **Page<*T*>** sur une de nos méthode de repository
- Ainsi qu'un argument **Pageable**

Une méthode custom « findByName » :

```
Page<Person> findByName (String name, Pageable pageable);
```

Repository

Liste Paginées

Utiliser des listes paginées :

- Exemple dans un controller et service :

Controller

```
@GetMapping
public Page<Person> getListPerson(
        @RequestParam(value = "pageNumber", defaultValue = "0") Integer pageNumber,
        @RequestParam(value = "pageSize", defaultValue = "20") Integer pageSize
) {
    return personService.findAll(PageRequest.of(pageNumber, pageSize));
}
```

```
public Page<Person> findAll(Pageable pageable) {
    return personRepository.findAll(pageable);
}
```

Service

Spring Framework – REST Partie Bonus

Partie 3 – Gestion des exceptions avec @ResponseStatus

@ResponseStatus

- Par défaut, Spring retourne des erreurs 500, 404 ou 400 selon les exceptions qui surviennent.

- Lorsque l'on *throw* nous même une exception, c'est une 500 par défaut (RuntimeExceptions)

- Les erreurs 500 indiquent un fonctionnement anormal du serveur.

@ResponseStatus

- Parfois on veut indiquer à l'utilisateur qu'il a fait une erreur. Dans ce cas on ne renvoie pas une 500 mais plutôt une 400.

- On peut choisir l'exception renvoyée en utilisant **@ResponseStatus** sur les exceptions perso.

Exception qui retourne un NOT_FOUND (erreur 404) lorsqu'elle est lancée

```
@ResponseStatus(HttpStatus.NOT_FOUND)
public class NotFoundException extends RuntimeException {
}
```

Exception

@ResponseStatus

- Avec un message personnalisable :

Exception avec un message par défaut (« Mauvaise requête ») qui renvoie une erreur 400

Exception

```
@ResponseStatus(HttpStatus.BAD_REQUEST)
public class BadRequestException extends RuntimeException {
    public BadRequestException() {
        super("Mauvaise requête");
    }
    public BadRequestException(String message) {
        super(message);
    }
}
```

@ResponseStatus

- Avec un message par défaut :

Exception avec un message par défaut (« Mauvaise requête ») qui renvoie une erreur 400

Exception

```
@ResponseStatus(HttpStatus.BAD_REQUEST)
public class EntityNotFoundException extends RuntimeException {
    public EntityNotFoundException() {
        super("Entité non trouvée.");
    }
}
```

Spring Framework – REST Partie Bonus

Partie 4 – ResponseEntity et Gestion d’Exceptions

ResponseEntity

- L'objet **ResponseEntity<T>** permet de manipuler la réponse HTTP renvoyée par Spring dans les controllers.
- On remplace le type de retour par cet objet (objet typé)

Exemple avec le « new » de ResponseEntity

Controller

```
@PostMapping
public ResponseEntity<Animal> create(@RequestBody @Valid Animal animalToCreate) {
    return new ResponseEntity<>(
        animalService.create(animalToCreate), HttpStatus.CREATED);
}
```

ResponseEntity

- ResponseEntity.ok

Exemple avec méthodes statiques ResponseEntity.ok, ResponseEntity.created, etc

Controller

```
@GetMapping
public ResponseEntity<Page<Animal>> findAllPageable(
        @RequestParam Integer pageNumber,
        @RequestParam Integer pageSize
) {
    return ResponseEntity.ok(
            animalService.findAllPageable(PageRequest.of(pageNumber, pageSize))
    );
}
```

ResponseEntity

➡ Avec réponse « erreur » :

Exemple avec méthodes statiques ResponseEntity.ok, ResponseEntity.created, etc

Controller

```
@PutMapping("/{id}")
    public ResponseEntity<?> update(
            @PathVariable("id") Integer id,
            @RequestBody @Valid Animal animalToUpdate) {

        if (!id.equals(animalToUpdate.getId())) {
            return ResponseEntity.badRequest()
                .body("Les ids ne correspondent pas");
        }

        return ResponseEntity.ok(animalService.update(animalToUpdate));
    }
```

ResponseEntity

En savoir plus :

- Lien Baeldung

@ControllerAdvice

@ControllerAdvice (et @RestControllerAdvice) :

- Se met sur une **classe**
- Permet la gestion **globale** des exception pour tous les controllers
- RestControllerAdvice permet d’omettre les @ResponseBody sur les méthodes (utile en REST)

@ControllerAdvice

Exemple :

```
@RestControllerAdvice
public class ExceptionHandlerControllerAdvice {

    @ExceptionHandler({jakarta.persistence.EntityNotFoundException.class})
    @ResponseStatus(code = HttpStatus.BAD_REQUEST)
    public ErrorDto handleExceptionNotFound(Exception exception, WebRequest
request) {
        exception.printStackTrace();
        return new ErrorDto(
                HttpStatus.BAD_REQUEST.value(),
                LocalDateTime.now(),
                exception.getMessage(),
                request.getDescription(false)
        );
    }
}
```

@ControllerAdvice

Explication des annotations

```
@RestControllerAdvice
public class ExceptionHandlerControllerAdvice  {

    @ExceptionHandler({jakarta.persistence.EntityNotFoundException.class})
    @ResponseStatus(code = HttpStatus.BAD_REQUEST)
    public ErrorDto handleExceptionNotFound(Exception exception, WebRequest
request) {
        exception.printStackTrace();
        return new ErrorDto(
                HttpStatus.BAD_REQUEST.val
                LocalDateTime.now(),
                exception.getMessage()
                request.getDescription(
        );
    }
}
```

- L'exception qui va être « **interceptée** »
- Le code de réponse qui va être renvoyé

@ControllerAdvice

En cas d'exception, on renvoie un « ErrorDto » que l'on crée

```
@RestControllerAdvice
public class ExceptionHandlerControllerAdvice  {

    @ExceptionHandler({jakarta.persistence.EntityNotFoundException.class})
    @ResponseStatus(code = HttpStatus.BAD_REQUEST)
    public ErrorDto handleExceptionNotFound(Exception exception, WebRequest
request) {
        exception.printStackTrace();
        return new ErrorDto(
                HttpStatus.BAD_REQUEST.value(),
                LocalDateTime.now(),
                exception.getMessage(),
                request.getDescription(false)
        );
    }
}
```

Le DTO et sa construction qui est renvoyé au client

@ControllerAdvice

Notre « ErrorDto »

```
public class ErrorDto {
    private final int statusCode;
    private final LocalDateTime localDateTime;
    private final String message;
    private final String description;

    // Constructeur avec tous les champs
    // {...}

    // Getters (pas besoin de setters)
    // {...}

}
```

@ControllerAdvice

Les paramètres de la méthode :

```
@RestControllerAdvice
public class ExceptionHandlerControllerAdvice  {

    @ExceptionHandler({jakarta.persistence.EntityNotFoundException.class})
    @ResponseStatus(code = HttpStatus.BAD_REQUEST)
    public ErrorDto handleExceptionNotFound(Exception exception, WebRequest
request) {
        exception.printStackTrace();
        return new ErrorDto(
                HttpStatus.BAD_REQUEST.value(),
                LocalDateTime.now(),
                exception.getMessage(),
                request.getDescription(false)
        );
    }
}
```

- L'exception qui a été interceptée
- La requête qui a été faite

@ControllerAdvice

Résultat dans Postman :

Body Cookies Headers (4) Test Results Status: 400 Bad Request

Pretty Raw Preview Visualize JSON

```
1 {
2     "statusCode": 400,
3     "localDateTime": "2023-06-07T10:34:19.0534646",
4     "message": "L'entité non trouvée.",
5     "description": "uri=/rest/person/123213"
6 }
```

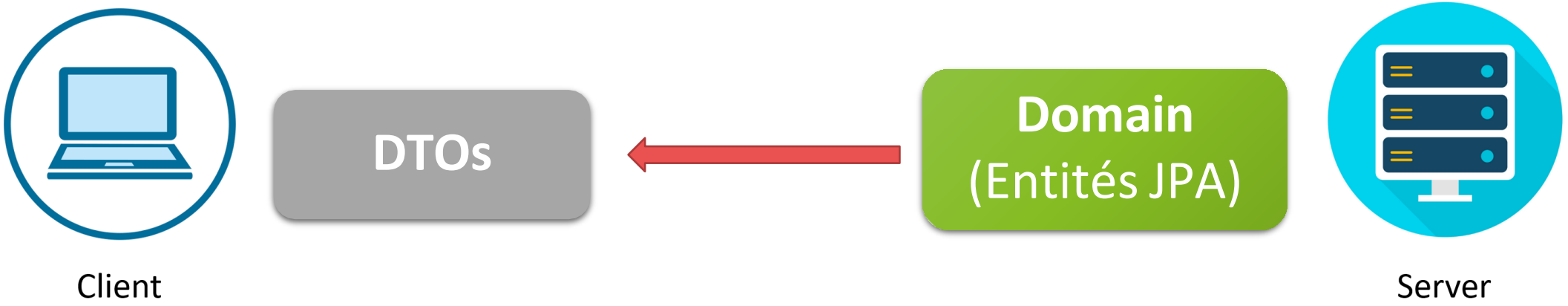
Spring Framework – REST Partie Bonus

Partie 5 – Utiliser des DTOs

Présentation des DTOs

- « DTO » (Data Transfert Object) est un design pattern.

- Un DTO est une classe Java classique (POJO – Plain Old Java Object) avec **champs privés**, **getters/setters**.
- **Il s'utilise à la place des entités JPA pour transiter les informations au client.**



Présentation des DTOs

Ils permettent :

- De **découpler notre modèle** de ce qui est présenté
  - *Ne pas trop lier notre modèle à la couche présentation*

- **D'adapter ou d'alléger les informations** transitées via les requêtes HTTP
  - *Enlever des infos inutiles ou sensibles*

Création d'un DTO

Exemple d'une Entité « Person » :

```
@Entity
public class Person {
    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Integer id;

    private Integer age;
    private String firstname;
    private String lastname;
    private String password;

    // Getters, setters
```

Montrer « password » au client ? Bof bof !

Création d'un DTO

- Plutôt que de renvoyer Person tel quel, on va faire un DTO :

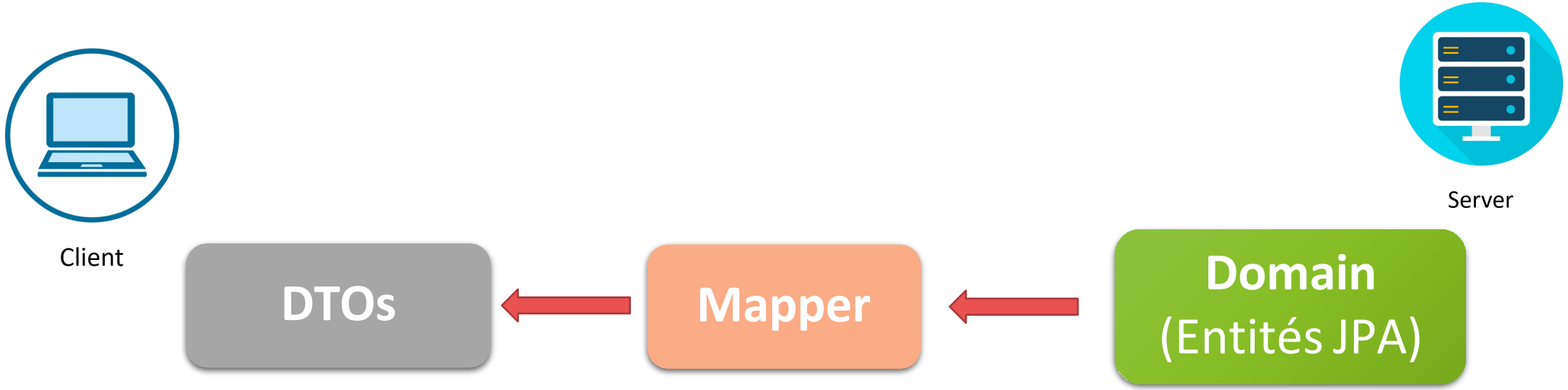
```
public class PersonLightDto {

    private Integer id;
    private Integer age;
    private String name;

    // Getters, Setters
```

- *On conserve l'ID car on en aura surement besoin*
- *Password n'y est plus*
- *Lastname et firstname ne sont plus que « Name »*

Création d'un DTO

Comment passer de l'un à l'autre ?

- On utilise de préférence **une classe « Mapper »**



Création d'un DTO

Comment passer de l'un à l'autre ?

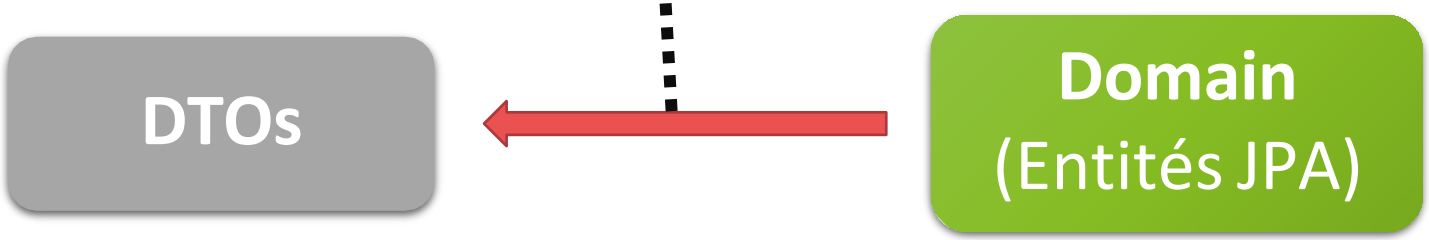
- Ou sinon **une méthode dans le DTO lui-même** si le cas est simple *(à éviter : Principe de Responsabilité Unique !)*

Exemple de méthodes :

```
PersonDto dto = new PersonDto(person);
Ou
PersonDto dto = PersonDto.toDto(person)
```

Client

Server



Création d'un DTO

Classe « Mapper » :

- Rôle : **transformer Animal en AnimalDTO** (et inversement selon les besoins)

- Sera un **bean spring @Component** (facilité d'utilisation)

Exemple d'un mapper pour DTO

Ex. d'un mapper pour notre PersonLightDto :

Mapper

```
@Component
public class PersonLightDtoMapper {

    // Transformer une Person en PersonLightDto
    public PersonLightDto toDto(Person person) {
        PersonLightDto dto = new PersonLightDto();
        dto.setId(person.getId());
        dto.setAge(person.getAge());
        dto.setName(
             person.getLastname() + " " + person.getFirstname());

        return dto;
    }
}
```

Une méthode toDto fait la transformation champ à champ !

Utiliser notre DTO et mapper

- Renvoyer des DTO dans les controllers et/ou services

Le controller renvoie des DTO

Controller

```
@GetMapping
public Page<PersonLightDto> getListPerson(
        @RequestParam Integer pageNumber,
        @RequestParam Integer pageSize
) {
    return personService.findAll(PageRequest.of(pageNumber, pageSize));
}
```

Utiliser notre DTO et mapper

- Renvoyer des DTO dans les controllers et/ou services

Service

```
@Autowired
private PersonLightDtoMapper personLightDtoMapper;

// { ... }


public Page<PersonLightDto> findAll(Pageable pageable) {
    Page<Person> pagePerson = personRepository.findAll(pageable);
    return pagePerson.map((person) -> personLightDtoMapper.toDto(person));
}
```

Ici on fait appel à la méthode « map » de Optional pour appeler la méthode du mapper sur l'entité contenue

Aller plus loin avec les « Mappers »

Il existe des lib tierces pour créer des mappers plus facilement :

- Mapstruct
- ModelMapper

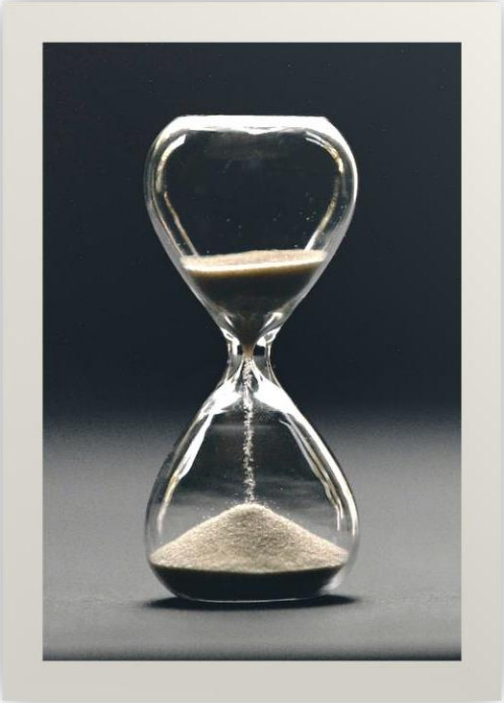
Spring Framework – REST Partie Bonus

Partie 6 – Les tâches cron

Les tâches cron

Cron ?

- Cron ← crontab ← **chrono tab**le
- Programme originaire des systèmes UNIX (1975) écrit en C



But :

- Lancer des **tâches routinières, périodiques, programmées dans le temps**
- Tâche cron ou « cronjob » en anglais

Les tâches cron

Comment ça marche ?

- **Les cronjobs se définissent selon une notation particulière**

```
┌─────────────── minute (0 - 59)
│ ┌───────────── hour (0 - 23)
│ │ ┌─────────── day of the month (1 - 31)
│ │ │ ┌───────── month (1 - 12)
│ │ │ │ ┌─────── day of the week (0 - 6) (Sunday to Saturday;
│ │ │ │ │                                7 is also Sunday on some systems)
│ │ │ │ │
│ │ │ │ │
* * * * * <command to execute>
```

Les tâches cron

Exemples :

- * * * * * = toutes les minutes, tous les jours du mois.
- 0 0 1 * * = tous les premiers du mois à 0h00min
- 0 0 * * 1 = tous les lundis à 0h00min
- 0/30 * * * = toutes les 30 minutes (utilisation du / « step value »)
  - *Pour s'exercer : crontab.guru*

Les tâches cron

Dans Spring :

- On active la fonctionnalité avec **@EnableScheduling**

```
@SpringBootApplication
@EnableScheduling
public class SpringBootApplication {
    public static void main(String[] args) {
        SpringApplication.run(SpringBootApplication.class, args);
    }
}
```

Classe principale
(ou classe de configuration)

Les tâches cron

Dans Spring :

- Le principe est repris, mais **ajoute en plus les secondes**
  - *(6 valeurs à mettre en tout)*
- Utilisation de **@Scheduled**

Service

```
@Scheduled(cron = "* * * * * *")
public void cron() {
    System.out.println("\n>>>>> CRON <<<< \n");
}
```

Cette méthode s'exécutera toutes les secondes.

Les tâches cron

Fonctionnalités en plus dans Spring :

- **fixedRate** et **fixedDelay**

Service

```
@Scheduled(fixedRate = 5000)
public void cron() {
    System.out.println("\n>>>>> CRON <<<< \n");
}
```

Cette méthode s'exécutera toutes 5 secondes.

Spring Framework – REST Partie Bonus

Partie 7 – Fichier application.properties

Fichier application.properties

On a déjà utilisé application.properties pour régler certaines valeurs pour Spring :

```
server.port=8082

# Datasource
spring.datasource.url=jdbc:mysql://localhost:3306/test2
spring.datasource.username=root
spring.datasource.password=root
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver

# Hibernate & JPA settings
spring.jpa.hibernate.ddl-auto=validate
spring.jpa.show-sql=true
```

Des paramètres utilisés par Spring dans application.properties

Fichier application.properties

Il est aussi possible d'y mettre **nos propres valeurs** :

```
# Mes valeurs personnalisées
mot.de.passe.ultra.secret=toto
utilisateur.prenom=Claude
utilisateur.email= Claude
cron.test=*/10 * * * * *
```

Des « constantes » que l'on pourra récupérer dans notre appli

Fichier application.properties

Puis d'aller les chercher, notation **${*le.nom.de.variable*}**

```
// Valeur mise dans un champ privé
@Value("${utilisateur.prenom}")
private String prenom;

// Valeur directement utilisée dans une annotation
@Scheduled(cron = "${cron.test}" )
public void test() {
    System.out.println("Je me lance et je m'appelle " + prenom);
}
```

Fichier application.properties

Quelle utilité ?

- On peut dire à Spring de **choisir** tel ou tel fichier *.properties lors du build
- Adapter les valeurs selon **l'environnement**
  - *déploiement sur serveur, en local, en production…*
- Définir des constantes en dehors du code java

- *Fichier properties avec Spring* *(Baeldung)*

Références et Ressources

@RequestParam

- Spring et @RequestParam (Baeldung)

Listes paginées

- Pagination and Sorting Spring Data JPA (Baeldung)

@ResponseStatus

- Spring et ResponseStatus (Baeldung)

ResponseEntity dans Spring

- Spring et ResponseEntity (Baeldung)

Gestion des exception (@ControllerAdvice) :

- Exception handling for REST in Spring : Section @ControllerAdvice (Baeldung)

Références et Ressources

DTOs

- DTO Pattern (Martin Fowler)
- DTOs dans Spring avec ModelMapper (Baeldung)
- Mapstruct et ModelMapper

Crons

- Crontab.guru
- Cron (page Wikipedia)
- Javadoc Spring CronExpression

Application.properties

- Doc Spring – Common Application Properties